

FULL DETAILED PREWORK SUBMISSION — FINANCE TRACKER (MERN Stack Project)

SECTION 1: CONTEXT

Finance Tracker is a full MERN stack application for managing expenses, incomes, budgets, recurring transactions, and receipts. Originally built in Python, the backend was migrated to Node.js for unified JavaScript development and improved deployment. MongoDB stores transactions, categories, and receipt metadata, while React provides a responsive UI. The system includes JWT authentication, file uploads, date-based filtering, category summaries, and optimized aggregation pipelines.

Primary Technical Constraints:

- Async file upload handling for receipt images.
- Efficient filtering and pagination for large datasets.
- Aggregation-heavy monthly/annual summaries.
- Secure token-based authentication.
- React performance optimization for large lists.

SECTION 2: TECHNICAL IMPLEMENTATION

Architecture:

React (Frontend, Vercel) → Express API (Backend, Render) → MongoDB (Atlas)
Cloud storage handles asynchronous receipt uploads. JWT middleware protects routes.

Critical Function — `addTransaction()`:

Handles validation, cloud upload, MongoDB insert, and error handling.

Data Flow:

React form → Axios request + JWT → Express validation → Cloud upload → MongoDB insert → Response → UI update.

SECTION 3: TECHNICAL DECISIONS

Decision 1: Migrated Python backend to Node.js

- Pros: unified JS stack, faster iteration, async-friendly
- Cons: required full rewrite

Decision 2: Chose MongoDB over SQL

- Pros: flexible schema, easier aggregation
- Cons: lacks strict relationships

Scaling Bottleneck:

Slow monthly summaries → fixed using compound indexes, caching, and simplified aggregation.

SECTION 4: LEARNINGS & ITERATION

Mistake:

Used synchronous uploads → blocked event loop.

Fix:

Switched to asynchronous streaming uploads.

Future Improvements:

Add Redis caching and background workers (BullMQ) for heavy tasks.